

Customizando o pipeline do spaCy

Trabalhando com Texto em Python I · Unidade 3

CiberExt 26-29 · FEELT38103 · Universidade Federal de Uberlândia

2026

Sumário

1. Modificando pipelines do spaCy	1
2. Processando textos com eficiência	3
3. Adicionando atributos personalizados aos objetos do spaCy	4
4. Gravando textos processados em disco	6
5. Simplificando a saída: sintagmas nominais e entidades nomeadas	8
Quiz	11
Resumo da unidade	11

Objetivos de aprendizagem

Ao final desta unidade, você deverá saber:

- **examinar e modificar** o pipeline do spaCy;
- **processar textos com eficiência**;
- **adicionar atributos personalizados** a objetos do spaCy;
- **salvar em disco** os textos processados;
- **mesclar** sintagmas nominais e entidades nomeadas em tokens únicos.

Vamos começar importando a biblioteca spaCy e o módulo `displacy`, usado para desenhar árvores de dependência, e carregando um modelo de linguagem para o inglês:

```
# Importa a biblioteca spaCy e o modulo displacy
from spacy import displacy
import spacy

# Carrega um modelo pequeno de lingua inglesa e o atribui a variavel 'nlp'
nlp = spacy.load('en_core_web_sm')

# Chama a variavel para examinar o objeto
nlp
```

```
<spacy.lang.en.English at 0x294522f10>
```

1. Modificando pipelines do spaCy

O objeto `Language` é, essencialmente, um **pipeline** que aplica um modelo de linguagem ao texto, executando as tarefas para as quais o modelo foi treinado. As tarefas realizadas dependem dos **componentes** presentes no pipeline.

Podemos examinar os componentes de um pipeline usando o atributo `pipeline` de um objeto `Language`:

```
nlp.pipeline
```

```
[('tok2vec', <spacy.pipeline.tok2vec.Tok2Vec at 0x294cf3dc0>),  
(('tagger', <spacy.pipeline.tagger.Tagger at 0x294cf3ca0>),  
(('parser', <spacy.pipeline.dep_parser.DependencyParser at 0x294b566d0>),  
(('attribute_ruler', <spacy.pipeline.attributeruler.AttributeRuler at 0x295187680>),  
(('lemmatizer', <spacy.lang.en.lemmatizer.EnglishLemmatizer at 0x295191540>),  
(('ner', <spacy.pipeline.ner.EntityRecognizer at 0x294b56740>))]]
```

Isso retorna um objeto `SimpleFrozenList` do spaCy, composto por **tuplas** do Python com dois itens:

- o **nome** do componente (por exemplo, `tagger`);
- o **componente** que efetivamente executa a tarefa (por exemplo, `spacy.pipeline.tok2vec.Tok2Vec`).

Componentes como `tagger`, `parser`, `ner` e `lemmatizer` já devem ser familiares da unidade anterior. Há, porém, dois componentes que ainda não encontramos:

- `tok2vec` mapeia os tokens para suas representações numéricas (veremos essas representações na Parte III).
- `attribute_ruler` aplica regras definidas pelo usuário aos tokens — por exemplo, correspondências a um dado padrão linguístico — e adiciona essa informação ao token como um atributo, se solicitado.

Nota: a lista de componentes em `nlp.pipeline` **não** inclui o `Tokenizer`, porque todo texto precisa ser tokenizado para que qualquer processamento ocorra. Por isso, o `Tokenizer` fica no atributo `tokenizer` do objeto `Language`, e não no atributo `pipeline`.

1.1 Excluindo componentes para ganhar desempenho

É importante entender que **todo componente do pipeline tem um custo computacional**. Se você não precisa da saída de um componente, não deve incluí-lo no pipeline, pois o processamento ficará mais lento.

Para excluir um componente, forneça o argumento `exclude` com uma *string* (ou uma lista) contendo os nomes dos componentes a excluir, ao inicializar o objeto `Language` com a função `load()`:

```
# Carrega um modelo pequeno de lingua inglesa, mas exclui o reconhecimento de  
# entidades nomeadas ('ner') e a analise de dependencias sintaticas ('parser').  
nlp = spacy.load('en_core_web_sm', exclude=['ner', 'parser'])  
  
# Examina os componentes ativos no objeto Language 'nlp'  
nlp.pipeline
```

```
[('tok2vec', <spacy.pipeline.tok2vec.Tok2Vec at 0x29514d760>),  
(('tagger', <spacy.pipeline.tagger.Tagger at 0x296c7ba00>),  
(('attribute_ruler', <spacy.pipeline.attributeruler.AttributeRuler at 0x296c6bb40>),  
(('lemmatizer', <spacy.lang.en.lemmatizer.EnglishLemmatizer at 0x296c72200>))]]
```

Como mostra a saída, os componentes `ner` e `parser` **não** estão mais no pipeline.

1.2 Analisando o pipeline com `analyze_pipes()`

O objeto `Language` também fornece o método `analyze_pipes()`, que dá uma visão geral dos componentes e de suas interações. Definindo o atributo `pretty` como `True`, o `spaCy` imprime uma tabela com os componentes e as anotações que produzem:

```
# Analisa o pipeline e guarda a análise na variável 'pipe_analysis'
pipe_analysis = nlp.analyze_pipes(pretty=True)
```

```
===== Pipeline Overview =====

#   Component      Assigns      Requires      Scores      Retokenizes
-   -
0   tok2vec        doc.tensor
1   tagger         token.tag      tag_acc
2   attribute_ruler
3   lemmatizer     token.lemma    lemma_acc
False
False
False
False

✓ No problems found.
```

O método `analyze_pipes()` retorna um **dicionário** Python, com a mesma informação da tabela. Você pode usá-lo para verificar que nenhum problema foi encontrado **antes** de processar grandes volumes de dados. Os relatórios de problemas ficam guardados sob a chave `problems`:

```
# Examina o valor guardado sob a chave 'problems'
pipe_analysis['problems']
```

```
{'tok2vec': [], 'tagger': [], 'attribute_ruler': [], 'lemmatizer': []}
```

Isso retorna um dicionário com os nomes dos componentes como chaves, cujos valores são **listas de problemas**. Neste caso, as listas estão vazias, porque não há problemas.

Podemos escrever facilmente um trecho de código que **verifica** se isso é realmente verdade. Para isso, percorremos o dicionário `pipe_analysis` usando o método `items()` para obter os pares chave/valor, e usamos a instrução `assert` com a função `len()` e o operador de comparação `==` para checar que a lista tem tamanho `0`. Se essa afirmação **não** for verdadeira (ou seja, se houver algum problema), o Python levanta um `AssertionError` e para:

```
# Percorre os pares chave/valor do dicionário. Atribui a chave e o valor
# as variáveis 'component_name' e 'problem_list'.
for component_name, problem_list in pipe_analysis['problems'].items():

    # Usa 'assert' para checar a lista de problemas; levanta Error se necessario.
    assert len(problem_list) == 0, f"There is a problem with {component_name}:
    {problem_list}!"
```

Aqui também imprimimos uma mensagem de erro usando uma *f-string*: o `f` antes das aspas declara que a *string* pode ser formatada, permitindo **inserir variáveis** entre chaves `{}`. Se um erro for levantado, essas partes serão preenchidas com os valores atuais de `component_name` e `problem_list`. Se nenhum problema for encontrado, o laço passa silenciosamente.

2. Processando textos com eficiência

Ao trabalhar com grandes volumes de dados, é altamente desejável processá-los da forma mais eficiente possível. Para ilustrar as boas práticas, vamos definir um exemplo com uma **lista** de três frases da Wikipédia em inglês:

```
# Reinicializa o modelo de linguagem, pois precisamos da análise de
# dependências nas próximas seções.
nlp = spacy.load('en_core_web_sm')

# Define uma lista de frases de exemplo
sents = ["On October 1, 2009, the Obama administration went ahead with a Bush
→ administration program, increasing nuclear weapons production.",
        "The 'Complex Modernization' initiative expanded two existing nuclear sites to
→ produce new bomb parts.",
        "The administration built new plutonium pits at the Los Alamos lab in New Mexico
→ and expanded enriched uranium processing at the Y-12 facility in Oak Ridge,
→ Tennessee."]

# Chama a variável para examinar a saída
sents
```

Os objetos **Language** do spaCy têm um método específico, **pipe()**, para processar textos guardados em uma **lista**. Esse método foi otimizado para essa finalidade: ele processa os textos em **lotes** (*batches*), em vez de individualmente, o que o torna mais rápido do que processar cada item com um laço **for**.

O método **pipe()** recebe uma lista como entrada e retorna um **gerador** (*generator*) do Python:

```
# Alimenta a lista de frases ao método pipe()
docs = nlp.pipe(sents)

# Chama a variável para examinar a saída
docs
```

```
<generator object Language.pipe at 0x296d0b580>
```

Geradores são objetos do Python que contêm outros objetos. Quando chamado, um gerador **produz** (*yields*) os objetos contidos nele. Para recuperar todos os objetos de um gerador, precisamos convertê-lo em outro tipo, como uma **lista** — pense na lista como uma estrutura capaz de coletar a saída do gerador para exame:

```
# Converte o gerador pipe em uma lista
docs = list(docs)

# Chama a variável para examinar a saída
docs
```

Isso nos dá uma lista de objetos **Doc** do spaCy prontos para processamento posterior.

3. Adicionando atributos personalizados aos objetos do spaCy

A unidade anterior mostrou como acessar as anotações linguísticas do spaCy por meio de seus atributos. Além disso, o spaCy permite **definir atributos personalizados** para objetos **Doc**, **Span** e **Token**. Esses atributos podem guardar, por exemplo, informações adicionais sobre os textos — se você trabalha com textos

que trazem dados sobre os usuários da língua, pode incorporar essa informação diretamente nos objetos do spaCy.

Atributos personalizados são adicionados ao objeto `Doc` com o método `set_extension()`. Como esses atributos são adicionados a **todos** os objetos `Doc` (e não a um `Doc` individual), primeiro importamos o objeto `Doc` genérico do módulo `tokens` do spaCy:

```
# Importa o objeto Doc do modulo 'tokens' do spaCy
from spacy.tokens import Doc

# Adiciona dois atributos personalizados ao objeto Doc, 'age' e 'location',
# usando o metodo set_extension().
Doc.set_extension("age", default=None)
Doc.set_extension("location", default=None)
```

Usamos o argumento `default` para definir um valor padrão para ambos, com a palavra-chave `None` do Python.

Nota: diferentemente de atributos como `sents` ou `heads`, os atributos personalizados ficam sob um atributo que consiste no caractere sublinhado `_` — por exemplo, `Doc._.age`.

Para exemplificar, vamos definir um dicionário Python. O dicionário `sents_dict` tem três chaves (0, 1 e 2), cujos valores são, por sua vez, dicionários com três chaves: `age`, `location` e `text`. Isso mostra como as estruturas de dados do Python costumam ser **aninhadas**:

```
# Cria um dicionario cujos valores sao outros dicionarios
# com tres chaves: 'age', 'location' e 'text'.
sents_dict = {0: {"age": 23,
                  "location": "Helsinki",
                  "text": "The Senate Square is by far the most important landmark in
↪ Helsinki."},
              1: {"age": 35,
                  "location": "Tallinn",
                  "text": "The Old Town, for sure."},
              2: {"age": 58,
                  "location": "Stockholm",
                  "text": "Södermalm is interesting!"},
              }
```

Vamos percorrer o dicionário `sents_dict` para processar os exemplos e adicionar os atributos personalizados aos objetos `Doc` resultantes:

```
# Prepara uma lista vazia para guardar os textos processados
docs = []

# Percorre os pares de chave e valor do dicionario 'sents_dict'.
# Os pares chave/valor ficam disponiveis pelo metodo items().
# Chamamos essas chaves e valores de 'key' e 'data'; ou seja, usamos
# a variavel 'data' para se referir ao dicionario aninhado.
for key, data in sents_dict.items():
```

```

# Recupera o valor da chave 'text' do dicionario aninhado.
# Alimenta esse texto ao modelo em 'nlp' e guarda o resultado em 'doc'.
doc = nlp(data['text'])

# Recupera os valores de 'age' e 'location' do dicionario aninhado.
# Atribui esses valores aos atributos personalizados do objeto Doc.
# Lembre-se: atributos personalizados ficam sob o pseudo-atributo '_'!
doc._.age = data['age']
doc._.location = data['location']

# Acrescenta o Doc atual em 'doc' a lista 'docs'
docs.append(doc)

```

Isso produz uma lista de objetos `Doc`, guardada em `docs`. Vamos percorrê-la e imprimir cada `Doc` com seus atributos personalizados:

```

# Percorre cada objeto Doc na lista 'docs'
for doc in docs:

    # Imprime cada Doc e os atributos 'age' e 'location'
    print(doc, doc._.age, doc._.location)

```

```

The Senate Square is by far the most important landmark in Helsinki. 23 Helsinki
The Old Town, for sure. 35 Tallinn
Södermalm is interesting! 58 Stockholm

```

3.1 Filtrando dados com compreensão de lista

Os atributos personalizados podem ser usados, por exemplo, para **filtrar** os dados. Uma forma eficiente é a **compreensão de lista** (*list comprehension*), que avalia o conteúdo de uma lista existente e monta uma nova lista com base em algum critério — é como um laço `for` declarado “na hora”, entre colchetes `[]`. Ela tem três componentes:

1. a referência a `doc` à esquerda do `for` define o que será guardado na nova lista (no caso, o próprio objeto `Doc`);
2. o `for ... in` funciona como num laço comum, percorrendo os itens de `docs`;
3. o `if` define uma **condição**: só incluímos objetos `Doc` cujo atributo `age` tenha valor abaixo de 40.

```

# Usa uma compreensao de lista para filtrar os Docs cujo atributo
# 'age' tenha valor abaixo de 40.
under_forty = [doc for doc in docs if doc._.get('age') < 40]

# Chama a variavel para examinar a saida
under_forty

```

```

[The Senate Square is by far the most important landmark in Helsinki.,
The Old Town, for sure.]

```

Isso retorna uma lista com apenas dois objetos `Doc` que atendem ao critério.

4. Gravando textos processados em disco

Ao trabalhar com grandes volumes de textos, primeiro garanta que o pipeline produz o resultado desejado usando **poucos** textos. Quando tudo estiver funcionando, processe o conjunto completo e **salve o resultado**, pois processar grandes volumes consome tempo e recursos.

O spaCy oferece um tipo de objeto especial, o `DocBin`, para armazenar objetos `Doc` com suas anotações linguísticas:

```
# Importa o objeto DocBin do modulo 'tokens' do spaCy
from spacy.tokens import DocBin

# Inicializa um objeto DocBin e adiciona os Docs de 'docs'
docbin = DocBin(docs=docs)
```

Atenção: se você adicionou **atributos personalizados** a `Docs`, `Spans` ou `Tokens`, também é preciso definir o argumento `store_user_data` como `True` — por exemplo, `DocBin(docs=docs, store_user_data=True)`.

Podemos verificar que os três `Docs` entraram no `DocBin` examinando a saída do método `__len__()`:

```
# Obtem o numero de Docs no DocBin
docbin.__len__()
```

```
3
```

O método `add()` permite adicionar mais objetos `Doc` ao `DocBin`, se necessário:

```
# Define uma string, alimenta o modelo em 'nlp' e adiciona o
# Doc resultante ao objeto DocBin 'docbin'
docbin.add(nlp("Yet another Doc object."))

# Verifica que o Doc foi adicionado; o tamanho agora deve ser 4
docbin.__len__()
```

```
4
```

Depois de populado, o `DocBin` deve ser **gravado em disco** com o método `to_disk()`, que recebe um único argumento, `path`, definindo o caminho do arquivo. Vamos gravar o `DocBin` em um arquivo chamado `docbin.spacy`, no diretório `data`:

```
# Grava o objeto DocBin em disco
docbin.to_disk(path='data/docbin.spacy')
```

Para **carregar** um `DocBin` do disco, primeiro inicialize um `DocBin` vazio com `DocBin()` e use o método `from_disk()`:

```
# Inicializa um novo DocBin e usa 'from_disk' para carregar os dados do disco.
# Atribui o resultado a variavel 'docbin_loaded'.
docbin_loaded = DocBin().from_disk(path='data/docbin.spacy')

# Chama a variavel para examinar a saida
docbin_loaded
```

```
<spacy.tokens._serialize.DocBin at 0x2953a4c40>
```

Por fim, para acessar os objetos `Doc` guardados no `DocBin`, use o método `get_docs()`. Ele recebe um único argumento, `vocab`, que precisa do **vocabulário** de um objeto `Language` (guardado no atributo `vocab`) para reconstruir a informação armazenada no `DocBin`:

```
# Usa 'get_docs' para recuperar os Docs do DocBin, passando o
# vocabulário em 'nlp.vocab' para reconstruir os dados.
# Converte o gerador resultante em uma lista para exame.
docs_loaded = list(docbin_loaded.get_docs(nlp.vocab))

# Chama a variável para examinar a saída
docs_loaded
```

```
[The Senate Square is by far the most important landmark in Helsinki.,
The Old Town, for sure.,
Södermalm is interesting!,
Yet another Doc object.]
```

Isso retorna uma lista com os quatro `Docs` adicionados ao `DocBin`.

Resumindo: o ideal é processar os textos **uma vez**, gravá-los em disco e carregá-los para as análises seguintes.

5. Simplificando a saída: sintagmas nominais e entidades nomeadas

5.1 Mesclando sintagmas nominais

Tarefas como anotação morfosintática e análise de dependências fazem previsões sobre **tokens individuais**. Às vezes, porém, é mais vantajoso operar com unidades linguísticas maiores, como **sintagmas nominais** (*noun phrases*) formados por vários tokens.

O spaCy dá acesso aos sintagmas nominais pelo atributo `noun_chunks` de um objeto `Doc`. Vamos imprimir os sintagmas de cada `Doc` na lista `docs`:

```
# Primeiro laço: percorre a lista 'docs'; 'doc' se refere aos itens da lista
for doc in docs:

    # Percorre cada sintagma nominal do objeto Doc
    for noun_chunk in doc.noun_chunks:

        # Imprime o sintagma nominal
        print(noun_chunk)
```

```
The Senate Square
the most important landmark
Helsinki
The Old Town
Södermalm
```

Para **mesclar** os sintagmas nominais em um único token, o spaCy oferece a função `merge_noun_chunks`, que pode ser adicionada ao pipeline com o método `add_pipe`:


```
# Adiciona o componente que mescla sintagmas nominais em Tokens unicos
nlp.add_pipe('merge_noun_chunks')
```

Nota: não é preciso reatribuir o objeto `Language` à mesma variável para atualizá-lo — o método `add_pipe` adiciona o componente automaticamente.

Processando novamente as três frases com `pipe()`, tudo **parece** igual (uma lista com três `Docs`). Mas, ao percorrer os tokens do primeiro `Doc` (`[0]`), vemos que os sintagmas nominais agora estão **mesclados** e rotulados como `NOUN`:

```
# Aplica o objeto Language 'nlp' a lista de frases em 'sents'
docs = list(nlp.pipe(sents))

# Percorre os Tokens do primeiro Doc da lista
for token in docs[0]:

    # Imprime o Token e sua classe gramatical
    print(token, token.pos_)
```

```
On ADP
October PROPN
1 NUM
, PUNCT
2009 NUM
, PUNCT
the Obama administration NOUN
went VERB
ahead ADV
with ADP
a Bush administration program NOUN
, PUNCT
increasing VERB
nuclear weapons production NOUN
. PUNCT
```

Rotular os sintagmas nominais como `NOUN` é uma aproximação razoável, já que suas palavras-núcleo são substantivos. Como a renderização com `displaCy` mostra, mesclar os sintagmas **simplifica** a árvore sintática:

```
displacy.render(docs[0], style='dep')
```

Embora os sintagmas nominais estejam agora representados por tokens únicos, eles continuam disponíveis no atributo `noun_chunks`. O `spaCy` os guarda como objetos `Span`, cujos atributos `start` e `end` determinam onde o `Span` começa e termina:

```
# Percorre os sintagmas nominais do primeiro Doc [0] na lista 'docs'
for noun_chunk in docs[0].noun_chunks:

    # Imprime o sintagma, seu tipo, e os índices onde começa e termina
    print(noun_chunk, type(noun_chunk), noun_chunk.start, noun_chunk.end)
```

```
October <class 'spacy.tokens.span.Span'> 1 2
the Obama administration <class 'spacy.tokens.span.Span'> 6 7
a Bush administration program <class 'spacy.tokens.span.Span'> 10 11
```

```
nuclear weapons production <class 'spacy.tokens.span.Span'> 13 14
```

5.2 Mesclando entidades nomeadas

Entidades nomeadas podem ser mescladas do mesmo modo, fornecendo `merge_entities` ao método `add_pipe()`. Primeiro, removemos a função `merge_noun_chunks` do pipeline com o método `remove_pipe()`:

```
# Remove a funcao 'merge_noun_chunks' do pipeline em 'nlp'
nlp.remove_pipe('merge_noun_chunks')

# Processa as frases originais novamente
docs = list(nlp.pipe(sents))
```

O método retorna uma tupla com o nome do componente removido e o próprio componente. Em seguida, adicionamos o componente `merge_entities`:

```
# Adiciona a funcao 'merge_entities' ao pipeline
nlp.add_pipe('merge_entities')

# Processa os dados novamente
docs = list(nlp.pipe(sents))

# Percorre os Tokens do terceiro Doc da lista
for token in docs[2]:

    # Imprime o Token e sua classe gramatical
    print(token, token.pos_)
```

```
The DET
administration NOUN
built VERB
new ADJ
plutonium NOUN
pits NOUN
at ADP
the DET
Los Alamos PROPN
lab NOUN
in ADP
New Mexico PROPN
and CCONJ
expanded VERB
enriched ADJ
uranium NOUN
processing NOUN
at ADP
the DET
Y-12 NUM
facility NOUN
in ADP
Oak Ridge PROPN
, PUNCT
Tennessee PROPN
```

Entidades nomeadas com vários tokens — como os topônimos “*Los Alamos*” e “*New Mexico*” — foram **mescladas** em tokens únicos.

Quiz

Marque a alternativa correta (a resposta certa está destacada com ✓).

1. Por que remover do pipeline componentes que você não usa?

1. Cada componente tem um custo computacional ✓
2. Deixa o código mais bonito
3. É obrigatório

2. Qual argumento de `load()` exclui componentes do pipeline?

1. `exclude` ✓
2. `remove`
3. `skip`

3. Por que o `Tokenizer` não aparece em `nlp.pipeline`?

1. Ele sempre roda e fica em `nlp.tokenizer` ✓
2. Foi removido do spaCy
3. É opcional e estava desligado

4. Onde ficam os atributos personalizados de um `Doc`?

1. Sob um atributo que consiste no sublinhado (`Doc._`) ✓
2. Numa coluna extra do texto
3. Dentro de `nlp.meta`

5. Qual método processa uma lista de textos em lote (mais rápido)?

1. `pipe()` ✓
2. `apply()`
3. `map()`

6. Para que serve o `DocBin`?

1. Guardar e gravar `Docs` processados em disco ✓
2. Baixar modelos de linguagem
3. Treinar o modelo

Resumo da unidade

Nesta unidade, você aprendeu a:

1. **examinar** o pipeline com `nlp.pipeline` e entender componentes como `tok2vec` e `attribute_ruler`;
2. **excluir componentes** (`exclude=[...]`) para ganhar desempenho e **auditar** o pipeline com `analyze_pipes()` e `assert`;
3. **processar textos em lote** com `nlp.pipe()`, entendendo geradores e a conversão para lista;

4. **adicionar atributos personalizados** (`set_extension()`, `Doc._.`) e **filtrar** dados com compreensão de lista;
5. **gravar e carregar** textos processados com `DocBin` (`to_disk()`, `from_disk()`, `get_docs()`);
6. **mesclar** sintagmas nominais (`merge_noun_chunks`) e entidades nomeadas (`merge_entities`) com `add_pipe()/remove_pipe()`.

Exercícios sugeridos

1. Carregue o modelo excluindo apenas o `lemmatizer` e confirme, com `nlp.pipeline`, que ele não está mais presente.
 2. Adicione um atributo personalizado `source` ao `Doc` e preencha-o com o nome do arquivo de origem ao processar textos da Unidade 1.
 3. Processe as notícias da Unidade 1 com `nlp.pipe()`, grave tudo em um `DocBin` e recarregue do disco, conferindo o número de `Docs`.
 4. Compare a árvore de dependências (`displacy`) de uma frase **com** e **sem** `merge_noun_chunks` e descreva a diferença.
-

Referências

- Rognvaldsson, E. et al. *Applied Language Technology* (MOOC). Universidade de Helsinque. <https://applied-language-technology.mooc.fi/>
- Honnibal, M.; Montani, I. et al. *spaCy: Industrial-strength Natural Language Processing in Python*. <https://spacy.io/>